Arduino - Potentiomètre Digital

1 Introduction

Un potentiomètre digital agit comme un potentiomètre classique, à l'exception du fait qu'il ne nécessite aucune interaction humaine ; il est contrôlé par un microcontrôleur.

Dans notre cas, nous utilisons le MCP41010, c'est un potentiomètre digital de 10kΩ qui possède 256 pas. Il utilise le protocole SPI (Serial Peripheral Interface) pour communiquer.

Ce circuit peut être utilisé dans divers circuits où il y a un besoin de potentiomètre, mais que l'on ne veut pas contrôler manuellement.

2 Pinout

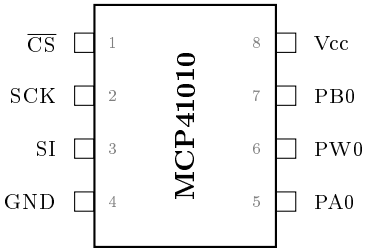

FIGURE 1 – Pinout du MCP41010

3 Branchement

La broche $\overline{CS}$ (Chip Select) doit être reliée à la broche 53 de l'Arduino Mega (ou à la broche 10 de l'Arduino UNO), elle permet de définir si la puce écoute (0) ou si elle n'écoute pas (1).

La broche SCK doit être reliée à la broche 52 de l'Arduino Mega (ou 13 sur un Arduino UNO), c'est l'horloge SPI.

La broche SI (ou MOSI) est la broche de signal SPI, elle doit être reliée à la broche 51 de l'Arduino Mega (ou 11 de l'Arduino Uno).

Les broches PA0, PB0 et PW0 sont les trois broches du potentiomètre, ici PA0 est reliée à l'alimentation positive, PB0 à la masse, et PW0 (le balai) est reliée à la broche A0 de l'Arduino (peu importe le type).

Les broches 4 et 8 doivent respectivement être branchées à la masse et au +5V. La figure est disponible à la page suivante (Figure 2).

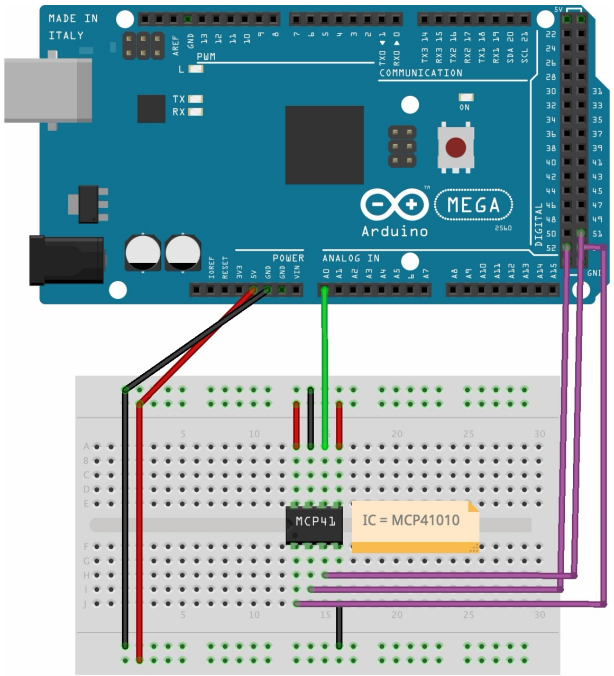

FIGURE 2 – Branchement à l'Arduino

4 Formules

$$R_{WA}(D_n) = \frac{(R_{AB})(256 - D_n)}{256} + R_W \tag{1}$$

$$R_{WB}(D_n) = \frac{(R_{AB})(D_n)}{256} + R_W \tag{2}$$

$$Resolution = \frac{R_{AB}}{256} = \frac{10k\Omega}{256} = 39\Omega \tag{3}$$

5 Programme

```
#include <MCP41xxx.h>                    // On inclut la librairie pour piloter la puce

#define WIPER A0                         // On définit la broche du balai

MCP41xxx digiPot(53);                    // On crée un objet

// Prototypes
int mvToRaw(int v);                      // Prototype de la fonction mvToRaw
float readTension(int pin);              // Prototype de la fonction readTension

void setup() {
  digiPot.begin();                       // On initialise la puce
  Serial.begin(9600);                    // On initialise la communication série à 9600 bauds
```

```
}

void loop() {
  digiPot.analogWrite(mvToRaw(1000));   // On demande une tension de 1000mV (1V)
  delay(1000);                          // On crée un délai d'une seconde
  Serial.print(readTension(WIPER));     // On lit la tension du balai
  Serial.println("V");                  // On affiche le symbole des volts
  digiPot.analogWrite(mvToRaw(2500));   // On demande une tension de 2500mV (2.5V)
  delay(1000);                          // On crée un délai d'une seconde
  Serial.print(readTension(WIPER));     // On lit la tension du balai
  Serial.println("V");                  // On affiche le symbole des volts
}

// Fonction qui permet de transformer une tension (en mV) en valeur 8 bits pour la puce
int mvToRaw(int v_mv) {
  int raw = (int)((float) v_mv * (255.0 / 5000.0));

  return constrain(raw, 0, 255);
}

// Fonction pour convertir la valeur 10 bits de l'ADC en tension
float readTension(int pin) {
  int valeurBrute = analogRead(pin);

  float tension = valeurBrute * (5.0 / 1024.0);

  return tension;
}
```